

# EEE 475/575 Medical Image Reconstruction & Processing

## Homework #5: l1-SPIRiT Reconstruction

Özgür Ekin Sönmez  
ID: 22201966

6 May 2026

### GenAI Usage Disclosure

OpenAI Codex based on GPT-5 was used to assist with MATLAB coding, figure/output organization, and LaTeX report drafting. The reconstruction implementation, parameter choices, results, and final submission were reviewed and are the responsibility of the student.

## 1 Overview

This homework implements compressed-sensing MRI reconstruction for an 8-channel Cartesian data set using random undersampling, wavelet-domain  $\ell_1$  regularization, SPIRiT interpolation, and iterative  $\ell_1$ -SPIRiT. All final sum-of-squares (SoS) images are normalized by their 95th percentile. The reference image is the SoS combination of the fully sampled coil images, and

$$\text{max\_val} = 2.1610$$

is the maximum pixel intensity of this normalized reference. For PSNR and SSIM, both the image of interest and the reference are divided by this same `max_val`.

## 2 Part 1: Fully Sampled Data

The magnitude image for each coil, the centered 2D Fourier spectra, and the SoS reference are shown below. The coil images have different spatial sensitivities, while the SoS image combines them into a single high-SNR reference.

## Part 1: Fully sampled coil magnitude images

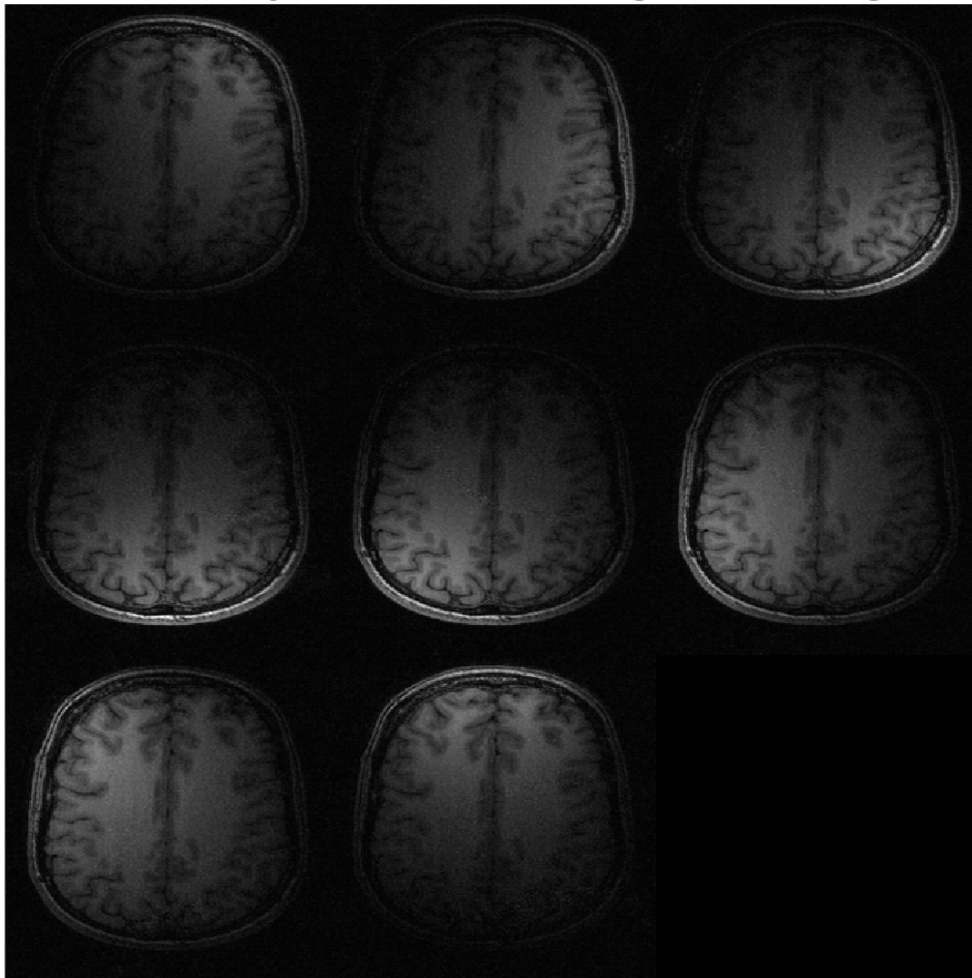


Figure 1: Magnitude images for all coils.

## Part 1: Fully sampled k-space spectra

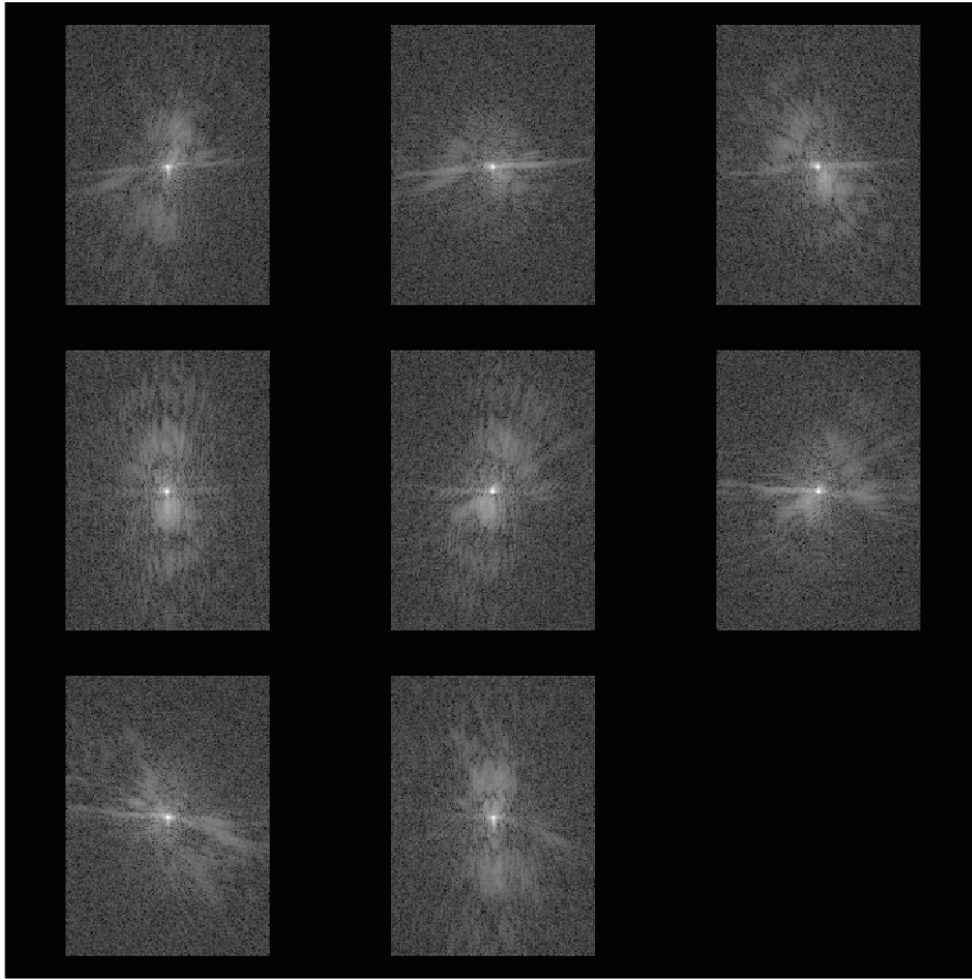


Figure 2: Log-scaled fully sampled k-space spectra for all coils.

## Part 1: SoS reference image

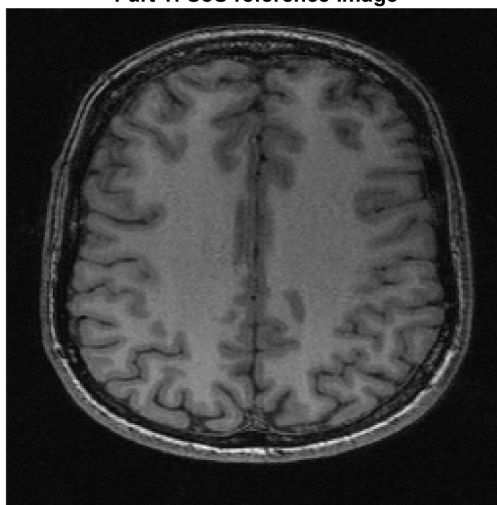


Figure 3: 95th-percentile-normalized SoS reference image.

**Relevant code.**  
1 | `kfull = fft2cStack(im);`

```

2
3 refSosRaw = sosCombine(im);
4 refSos = normalizeByP95(refSosRaw);
5 max_val = max(refSos(:));
6
7 saveMontageFigure(abs(im), [], fullfile(outDir, 'part1_coil_images.png'), ...
8     'Part1: Fully sampled coil magnitude images');
9 saveMontageFigure(log1p(abs(kfull)), [], fullfile(outDir, 'part1_kspace_spectra.png'), ...
10    'Part1: Fully sampled k-space spectra');
11 saveImageFigure(refSos, [0 max_val], fullfile(outDir, 'part1_sos_reference.png'), ...
12    'Part1: SoS reference image');

```

### 3 Part 2: Sampling Mask and Random Undersampling

The mask samples a fraction of k-space, giving an overall acceleration

$$R = \frac{1}{\text{sampled fraction}} = 4.958.$$

The zero-filled reconstruction directly applies the mask in k-space and transforms back to the image domain. Its measured quality is PSNR = 28.95 dB and SSIM = 0.8276.

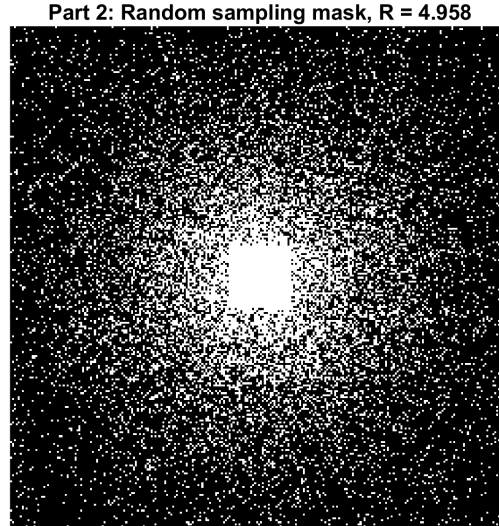


Figure 4: Random sampling mask with a central calibration region.

## Part 2: Random undersampled k-space spectra

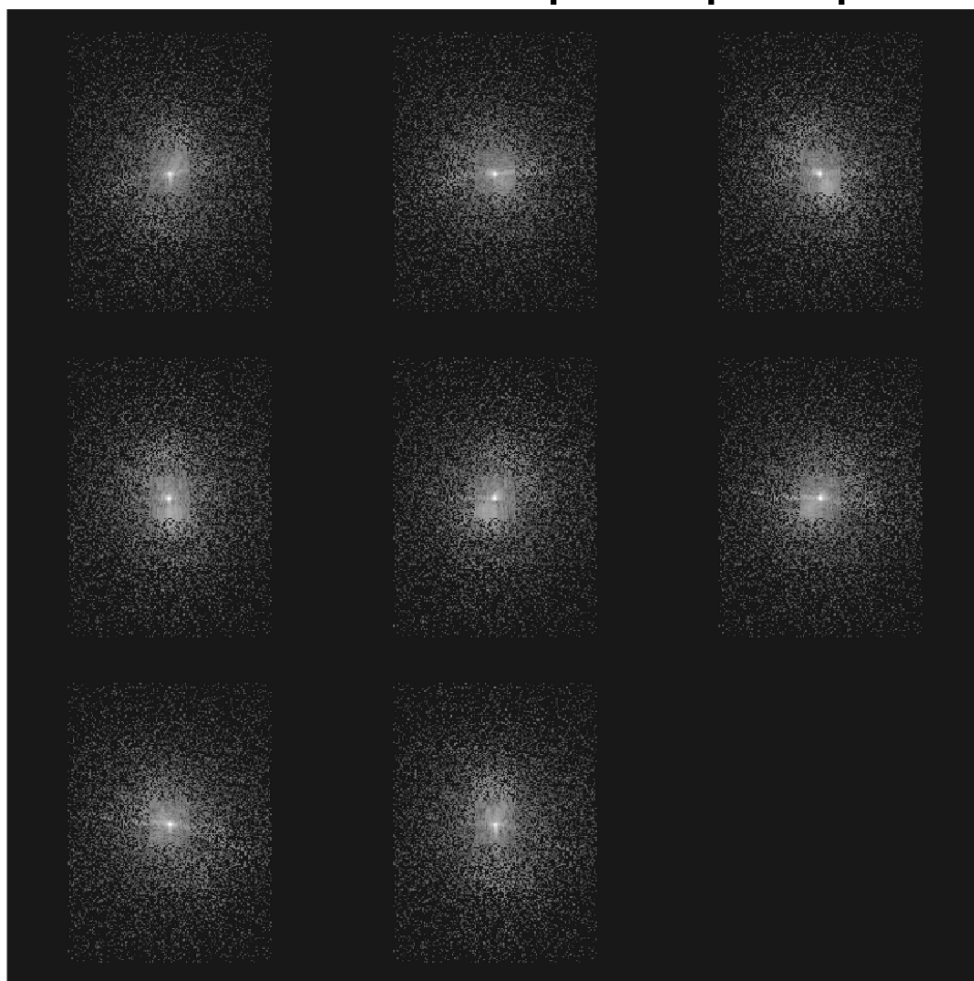


Figure 5: Log-scaled randomly undersampled k-space spectra.

## Part 2: Zero-fill coil magnitude images

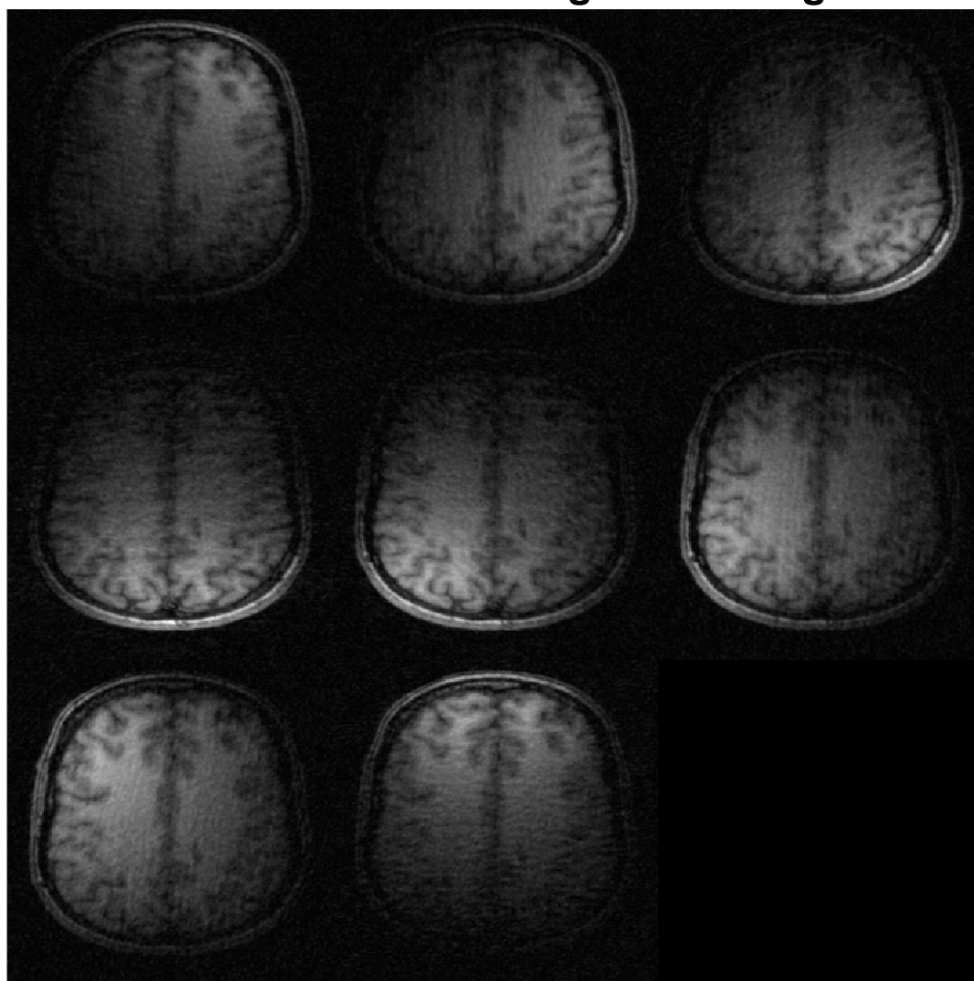
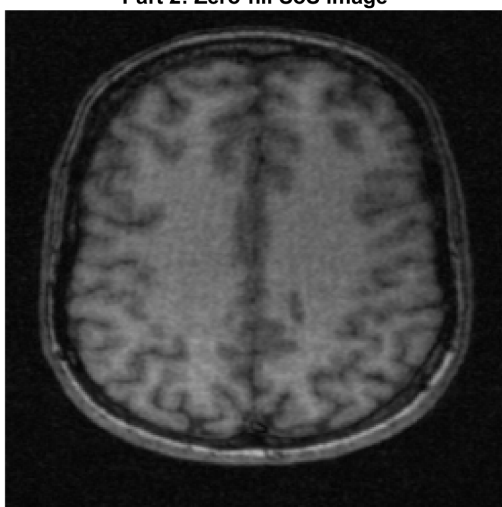


Figure 6: Zero-filled coil images.

Part 2: Zero-fill SoS image



Part 2: Zero-fill absolute error

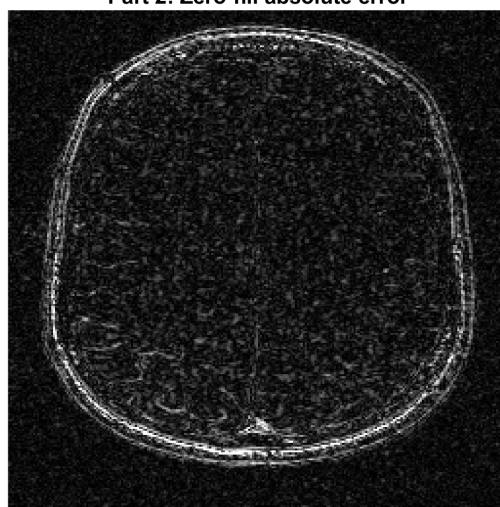


Figure 7: Zero-filled SoS image and absolute error image.

Compared with uniform undersampling, random undersampling spreads aliasing energy incoherently across the image. This produces noise-like artifacts rather than the coherent fold-over

artifacts seen when regularly skipping k-space lines.

#### Relevant code.

```

1 R = 1 / nnz(mask) * numel(mask);
2 [imu, Mu] = randundersample(im, mask);
3
4 zfSos = normalizeByP95(sosCombine(imu));
5 [psnrZF, ssimZF] = computeQuality(zfSos, refSos, max_val);
6 errZF = abs(zfSos - refSos);
7
8 saveImageFigure(double(mask), [0 1], fullfile(outDir, 'part2_mask.png'), ...
9     sprintf('Part2: Random sampling mask, R=%0.3f', R));
10 saveMontageFigure(log1p(abs(Mu)), [], fullfile(outDir, 'part2_kspace_spectra.png'), ...
11     'Part2: Random undersampled k-space spectra');
12 saveMontageFigure(abs(imu), [], fullfile(outDir, 'part2_zerofill_coil_images.png'), ...
13     'Part2: Zero-fill coil magnitude images');
14 saveImageFigure(zfSos, [0 max_val], fullfile(outDir, 'part2_zerofill_sos.png'), ...
15     'Part2: Zero-fill SoS image');
16 saveImageFigure(errZF, [0 0.2*max_val], fullfile(outDir, 'part2_zerofill_error.png'), ...
17     'Part2: Zero-fill absolute error');
18
19 function [imu, Mu] = randundersample(im, mask)
20     [Nx, Ny, Nc] = size(im);
21     Mu = zeros(Nx, Ny, Nc);
22     imu = zeros(Nx, Ny, Nc);
23     for c = 1:Nc
24         Mc = fft2c(im(:, :, c));
25         Mu(:, :, c) = Mc .* mask;
26         imu(:, :, c) = ifft2c(Mu(:, :, c));
27     end
28 end

```

## 4 Part 3: Wavelet-Domain $\ell_1$ Regularization

Wavelet sparsity is enforced with complex soft-thresholding. For a coefficient  $x$ , the threshold is  $\beta|x|_{\max}$  and the shrinkage rule is

$$S_{\beta}(x) = \frac{x}{|x|} \max(0, |x| - \beta|x|_{\max}).$$

Thresholding is applied to complex coil images, not to magnitude images.

### Part 3: Wavelet coefficient magnitudes before thresholding

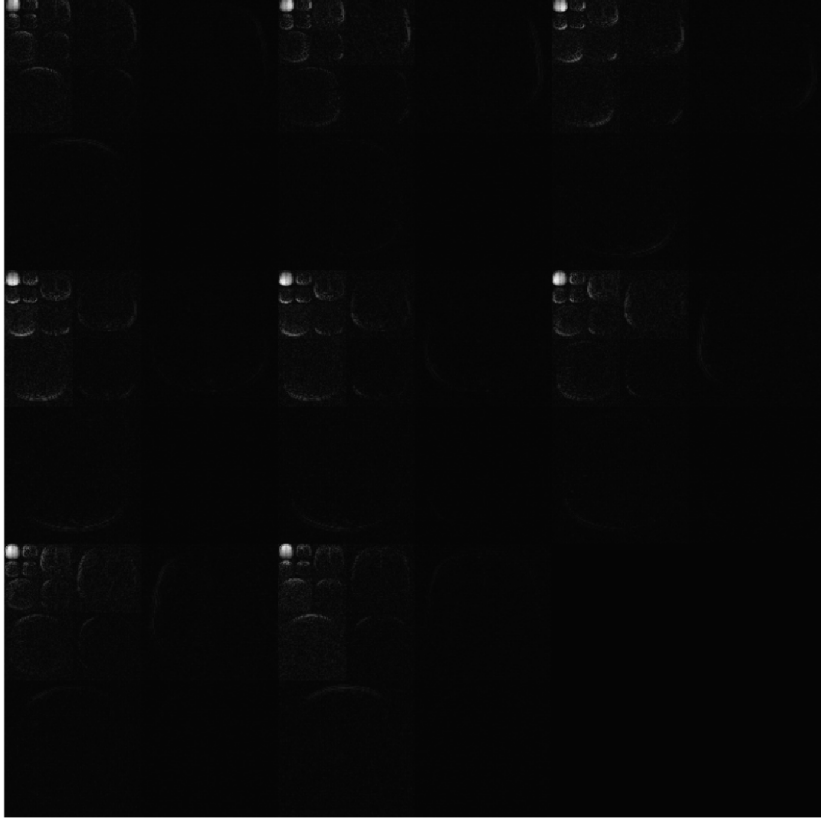


Figure 8: Log-scaled wavelet coefficient magnitudes before thresholding.

| $\beta$ | PSNR (dB) | SSIM   |
|---------|-----------|--------|
| 0.0001  | 28.94     | 0.8281 |
| 0.001   | 28.69     | 0.8258 |
| 0.01    | 25.96     | 0.6476 |
| 0.1     | 21.55     | 0.3836 |

Table 1: Wavelet-only regularization performance.

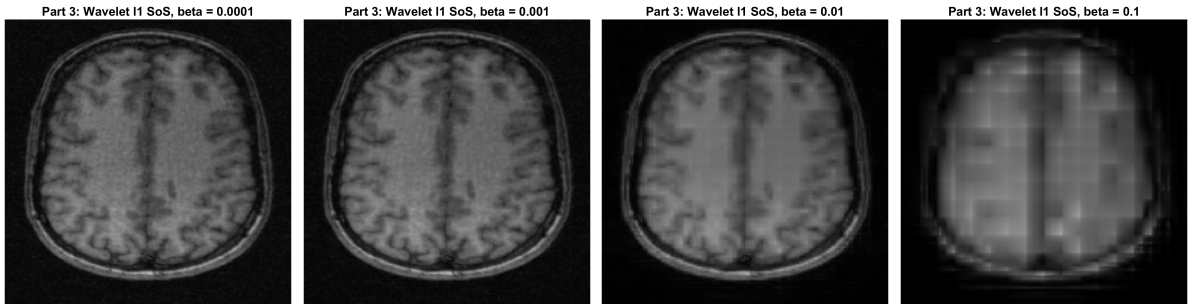


Figure 9: Wavelet-regularized SoS images for increasing  $\beta$ .

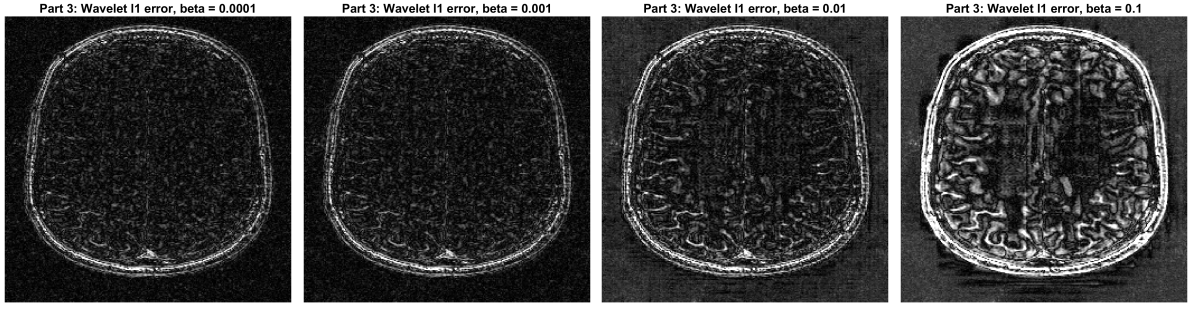


Figure 10: Wavelet-regularized absolute error images for increasing  $\beta$ .

Small  $\beta$  values preserve details but suppress less artifact energy. In this experiment,  $\beta = 0.0001$  gives the best SSIM among the first tested values and is very close to the zero-filled PSNR, meaning that the threshold is weak enough to avoid visible loss of anatomical structure. It may also be useful to test even smaller values such as  $10^{-5}$  or  $10^{-6}$ , because the current best low-threshold value is already the smallest value in the original sweep. Those lower values would behave almost like the zero-filled image with very mild wavelet shrinkage, and they could show whether a small amount of coefficient denoising improves SSIM without reducing PSNR.

I also tested the extra larger value  $\beta = 0.1$  to make the thresholding effect more obvious. This aggressive threshold removes many wavelet coefficients, so the SoS image is expected to look visibly over-smoothed and the error image should become much stronger. Larger  $\beta$  values remove more incoherent undersampling artifact, but excessive thresholding also removes anatomical detail and lowers image fidelity. This is already seen for  $\beta = 0.01$ , and it becomes clearer for the additional  $\beta = 0.1$  experiment.

#### Relevant code.

```

1 coeffBefore = zeros(Nx, Ny, Nc);
2 for c = 1:Nc
3     wv = Wavelet('Daubechies', 4, 4);
4     coeffBefore(:, :, c) = wv * imu(:, :, c);
5 end
6 saveMontageFigure(log1p(abs(coeffBefore)), [], ...
7     fullfile(outDir, 'part3_wavelet_coeff_before.png'), ...
8     'Part_3:_Wavelet_coefficient_magnitudes_before_thresholding');
9
10 for b = 1:numel(betaListPart3)
11     beta = betaListPart3(b);
12     imth = zeros(Nx, Ny, Nc);
13     for c = 1:Nc
14         imth(:, :, c) = l1wavelet(imu(:, :, c), beta);
15     end
16
17     thSos = normalizeByP95(sosCombine(imth));
18     [psnrTh, ssimTh] = computeQuality(thSos, refSos, max_val);
19     errTh = abs(thSos - refSos);
20 end
21
22 function imth = l1wavelet(imr, beta)
23     wv = Wavelet('Daubechies', 4, 4);
24     coeffW = wv * imr;
25     maxCoeff = max(abs(coeffW(:)));
26     if maxCoeff == 0
27         imth = imr;
28         return;
29     end
30     mag = abs(coeffW);
31     shrink = max(0, mag - beta * maxCoeff) ./ (mag + eps);
32     coeffWth = coeffW .* shrink;
33     imth = wv' * coeffWth;
34 end

```

## 5 Part 4: SPIRiT Kernel Weights

A  $3 \times 3$  SPIRiT kernel was calibrated from the central  $32 \times 32$  region of k-space. For each target coil, the center point of that coil is excluded, while all 8 neighboring samples from the target coil and all 9 samples from every other coil are used. Therefore, the kernel has  $(9N_c - 1) \times N_c = 71 \times 8$  complex weights.

**Part 4: SPIRiT kernel magnitude, lambda = 0**

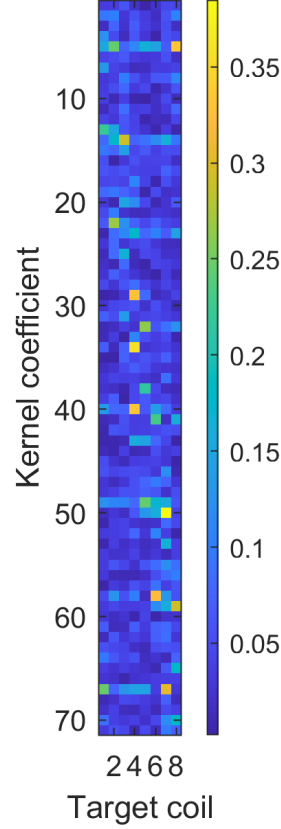


Figure 11: Magnitude of SPIRiT kernel weights for  $\lambda = 0$ .

For reconstruction, I used  $\lambda = 0.001$ . The kernel is sample-geometry independent, so the same calibrated weights are used at every k-space location and iteration.

### Relevant code.

```

1 Mc = extractCalibration(kfull, [32 32]);
2 kernel0 = calibrateSpirit(Mc, 0);
3 kernel = calibrateSpirit(Mc, lambdaSpirit);
4
5 saveKernelFigure(kernel0, fullfile(outDir, 'part4_kernel_lambda0.png'), ...
6     'Part_4:_SPIRiT_kernel_magnitude,_lambda_0');
7
8 function kernel = calibrateSpirit(Mc, lambda)
9     [Nx, Ny, Nc] = size(Mc);
10    offsets = -1:1;
11    nFeat = 9 * Nc - 1;
12    nRows = (Nx - 2) * (Ny - 2);
13    kernel = zeros(nFeat, Nc);
14
15    for targetCoil = 1:Nc
16        A = zeros(nRows, nFeat);
17        y = zeros(nRows, 1);
18        row = 0;
19        for ix = 2:Nx-1
20            for iy = 2:Ny-1

```

```

21         row = row + 1;
22         feat = 0;
23         for coil = 1:Nc
24             for dx = offsets
25                 for dy = offsets
26                     if coil == targetCoil && dx == 0 && dy == 0
27                         continue;
28                     end
29                     feat = feat + 1;
30                     A(row, feat) = Mc(ix + dx, iy + dy, coil);
31                 end
32             end
33         end
34         y(row) = Mc(ix, iy, targetCoil);
35     end
36 end
37
38 normalMat = A' * A;
39 if lambda > 0
40     regScale = trace(normalMat) / nFeat;
41     normalMat = normalMat + lambda * regScale * eye(nFeat);
42 end
43 kernel(:, targetCoil) = normalMat \ (A' * y);
44 end
45 end

```

## 6 Part 5: One SPIRiT Iteration Without Data Consistency

One SPIRiT iteration estimates every k-space sample from neighboring multi-coil samples. Data consistency is not enforced in this part, so even acquired points are replaced by SPIRiT estimates. The resulting image has PSNR = 28.27 dB and SSIM = 0.7933.

The SPIRiT kernel estimation solves a regularized least-squares problem on the  $32 \times 32$  calibration region. I used  $\lambda = 0.001$  for reconstruction. The role of  $\lambda$  is to stabilize the inversion of  $A^H A$  during kernel calibration. If  $\lambda$  is too small, the calibrated weights can overfit the calibration data and become sensitive to noise or local correlations in the ACS region. If  $\lambda$  is too large, the kernel weights are overly damped, which weakens the interpolation and leaves more residual undersampling artifact. The chosen value is small enough to preserve interpolation strength, but nonzero so that the kernel calibration is numerically more stable than the  $\lambda = 0$  case displayed in Part 4.

## Part 5: One SPIRiT iteration coil images

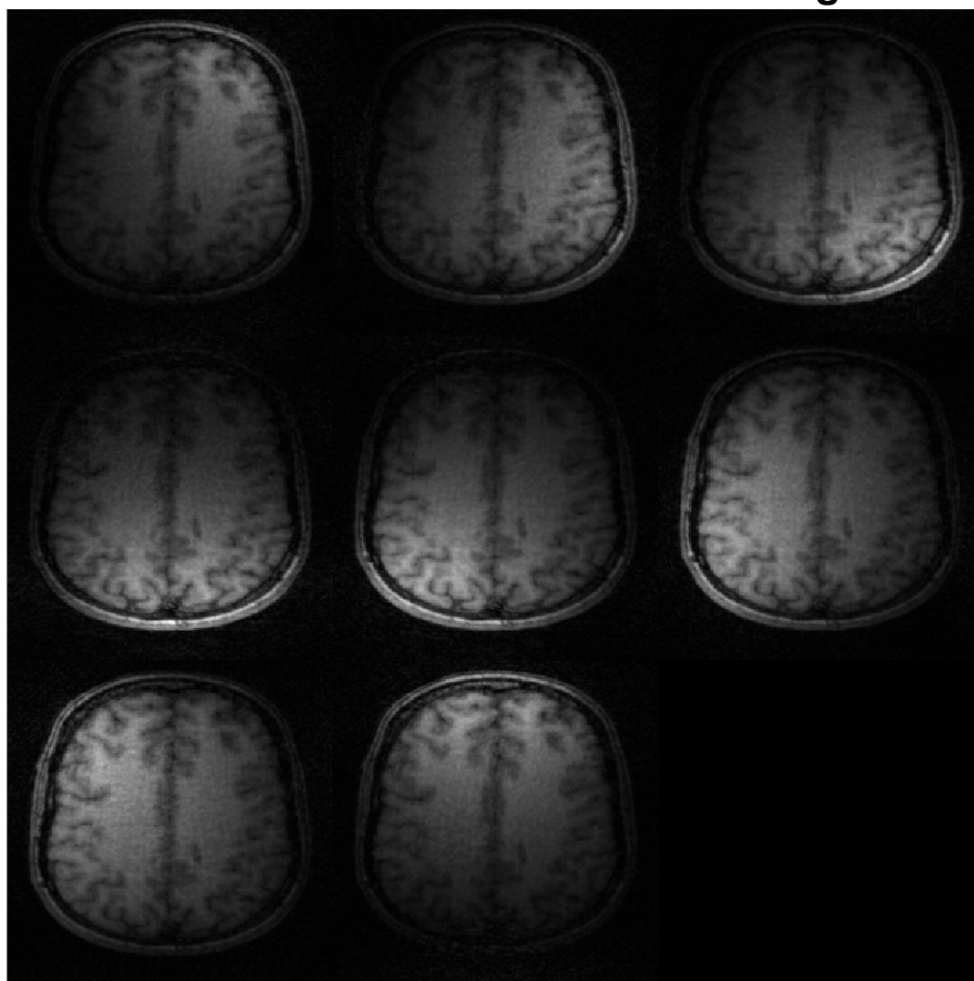


Figure 12: Coil images after one SPIRiT iteration without data consistency.

### Part 5: One SPIRiT iteration k-space spectra

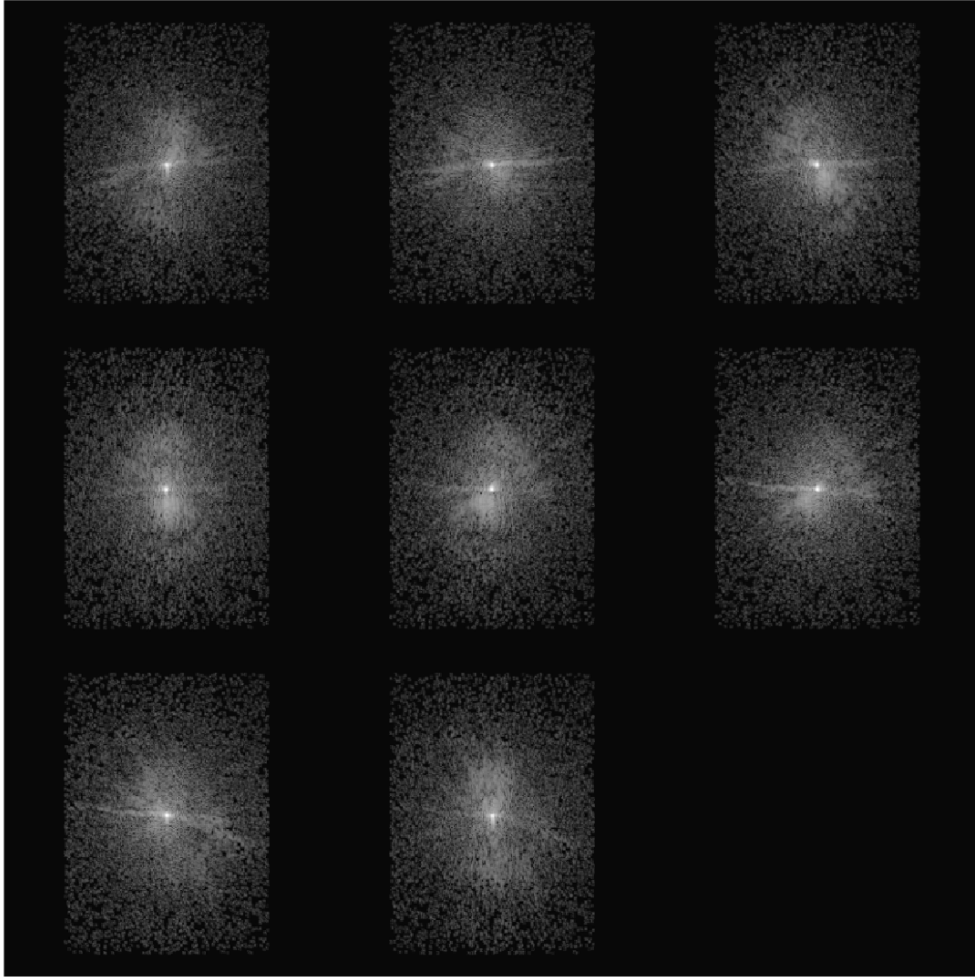
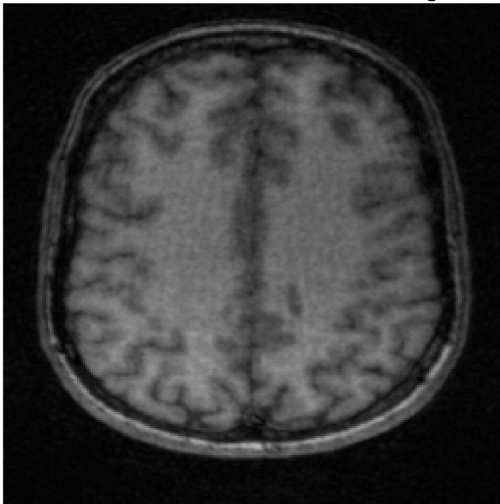


Figure 13: K-space spectra after one SPIRiT iteration without data consistency.

### Part 5: One SPIRiT iteration SoS image



### Part 5: One SPIRiT iteration absolute error

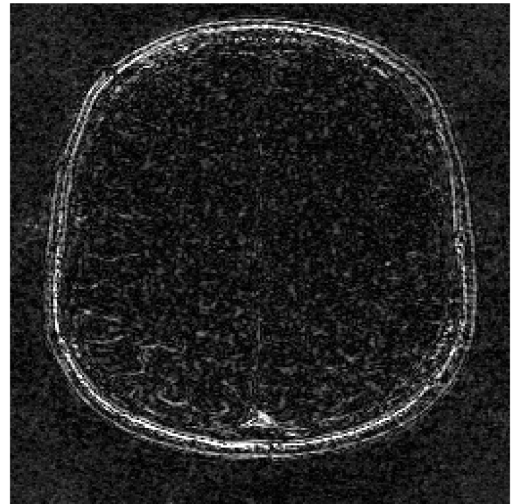


Figure 14: SoS image and error after one SPIRiT iteration without data consistency.

Without data consistency, the reconstruction can drift away from the measurements. The result is slightly worse than the zero-filled image because the SPIRiT operation replaces not only

missing k-space values but also originally acquired reliable samples. Therefore, this part should be interpreted mainly as the effect of applying the calibrated kernel alone. The next part adds data consistency, which is essential for using SPIRiT as a reconstruction algorithm rather than only as an interpolation operator.

#### Relevant code.

```

1 [imSpirit1, MSpirit1] = spirit(Mu, kernel);
2 spirit1Sos = normalizeByP95(sosCombine(imSpirit1));
3 [psnrSpirit1, ssimSpirit1] = computeQuality(spirit1Sos, refSos, max_val);
4 errSpirit1 = abs(spirit1Sos - refSos);
5
6 function [imr, Mr] = spirit(Mu, kernel)
7     [Nx, Ny, Nc] = size(Mu);
8     offsets = -1:1;
9     Mr = zeros(Nx, Ny, Nc);
10
11     for targetCoil = 1:Nc
12         feat = 0;
13         estimate = zeros(Nx, Ny);
14         for coil = 1:Nc
15             for dx = offsets
16                 for dy = offsets
17                     if coil == targetCoil && dx == 0 && dy == 0
18                         continue;
19                     end
20                     feat = feat + 1;
21                     neighbor = shift2dZero(Mu(:, :, coil), dx, dy);
22                     estimate = estimate + kernel(feat, targetCoil) * neighbor;
23                 end
24             end
25         end
26         Mr(:, :, targetCoil) = estimate;
27     end
28
29     imr = ifft2cStack(Mr);
30 end

```

## 7 Part 6: One Full SPIRiT Iteration With Data Consistency

After SPIRiT estimation, measured k-space points are restored from the acquired data. This full iteration gives PSNR = 30.63 dB and SSIM = 0.8566.

## Part 6: One full SPIRiT iteration coil images

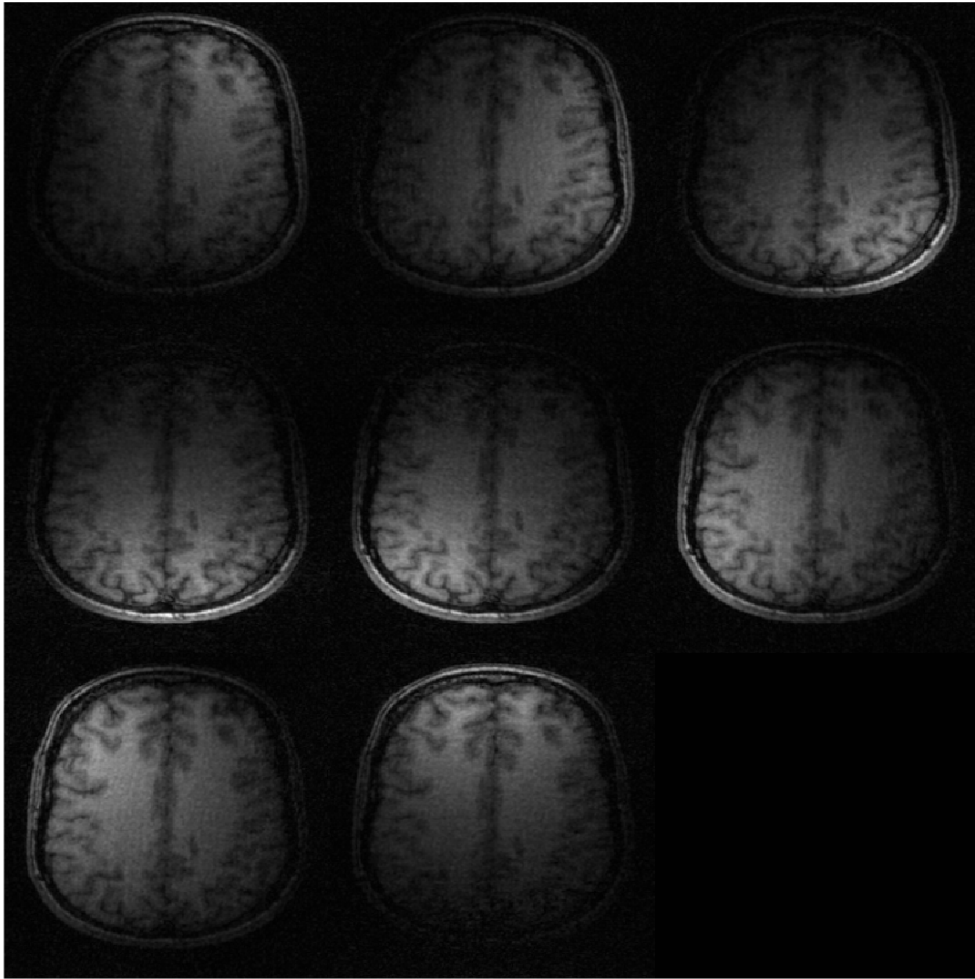


Figure 15: Coil images after one full SPIRiT iteration with data consistency.

### Part 6: One full SPIRiT iteration k-space spectra

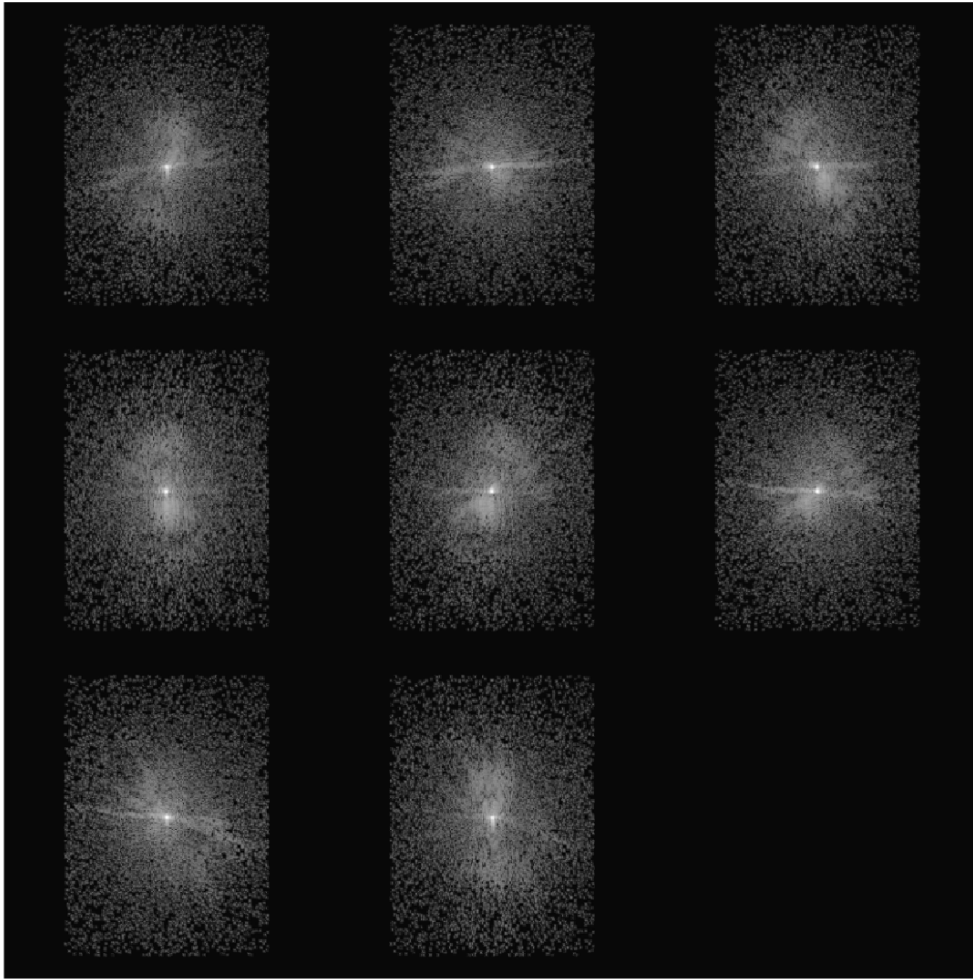
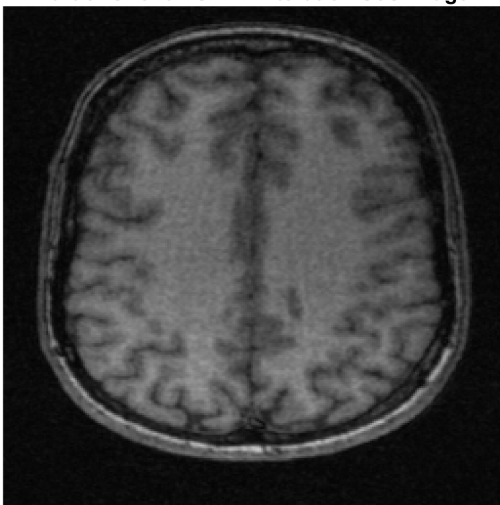


Figure 16: K-space spectra after one full SPIRiT iteration with data consistency.

### Part 6: One full SPIRiT iteration SoS image



### Part 6: One full SPIRiT iteration absolute error

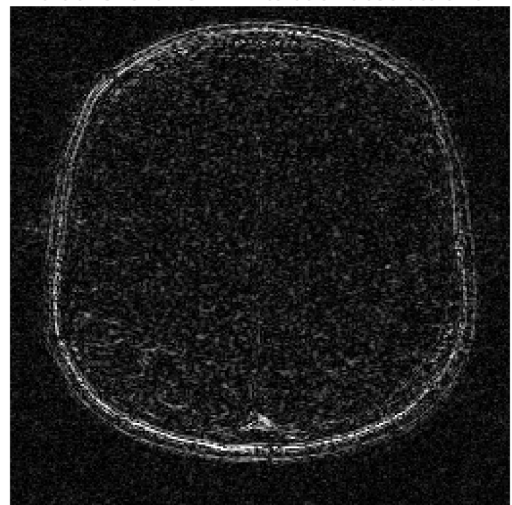


Figure 17: SoS image and error after one full SPIRiT iteration with data consistency.

Data consistency prevents the reconstruction from overwriting known samples and improves stability relative to the no-DC case. The improvement from Part 5 to Part 6 is clear: PSNR

increases from 28.27 dB to 30.63 dB, and SSIM increases from 0.7933 to 0.8566. This shows that the SPIRiT kernel is useful for estimating missing locations, but the measured k-space samples must remain fixed. Visually, the error image is also more concentrated around residual high-frequency differences rather than being spread broadly by replacing acquired data.

#### Relevant code.

```

1 MFull1 = applyDataConsistency(MSpirit1, Mu, mask);
2 imFull1 = ifft2cStack(MFull1);
3 full1Sos = normalizeByP95(sosCombine(imFull1));
4 [psnrFull1, ssimFull1] = computeQuality(full1Sos, refSos, max_val);
5 errFull1 = abs(full1Sos - refSos);
6
7 function Mdc = applyDataConsistency(Mr, Mu, mask)
8     Mdc = Mr;
9     for c = 1:size(Mr, 3)
10         tmp = Mdc(:, :, c);
11         acquired = Mu(:, :, c);
12         tmp(mask) = acquired(mask);
13         Mdc(:, :, c) = tmp;
14     end
15 end

```

## 8 Part 7: Ten SPIRiT Iterations

The SPIRiT kernel is calibrated once and then reused for all 10 iterations. At each iteration, SPIRiT estimation is followed by data consistency. The last iteration gives PSNR = 32.91 dB and SSIM = 0.8683.

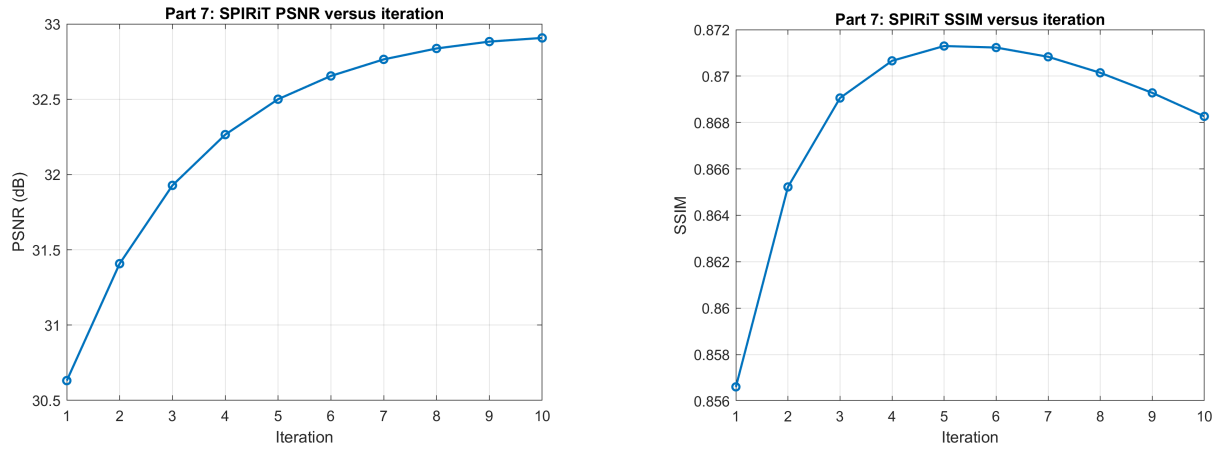


Figure 18: PSNR and SSIM versus SPIRiT iteration number.

| Iteration | PSNR (dB) | SSIM   |
|-----------|-----------|--------|
| 1         | 30.63     | 0.8566 |
| 2         | 31.41     | 0.8652 |
| 3         | 31.93     | 0.8691 |
| 4         | 32.27     | 0.8707 |
| 5         | 32.50     | 0.8713 |
| 6         | 32.65     | 0.8712 |
| 7         | 32.76     | 0.8708 |
| 8         | 32.84     | 0.8701 |
| 9         | 32.88     | 0.8693 |
| 10        | 32.91     | 0.8683 |

Table 2: Quantitative results for 10 full SPIRiT iterations.

### Part 7: Last SPIRiT iteration coil images

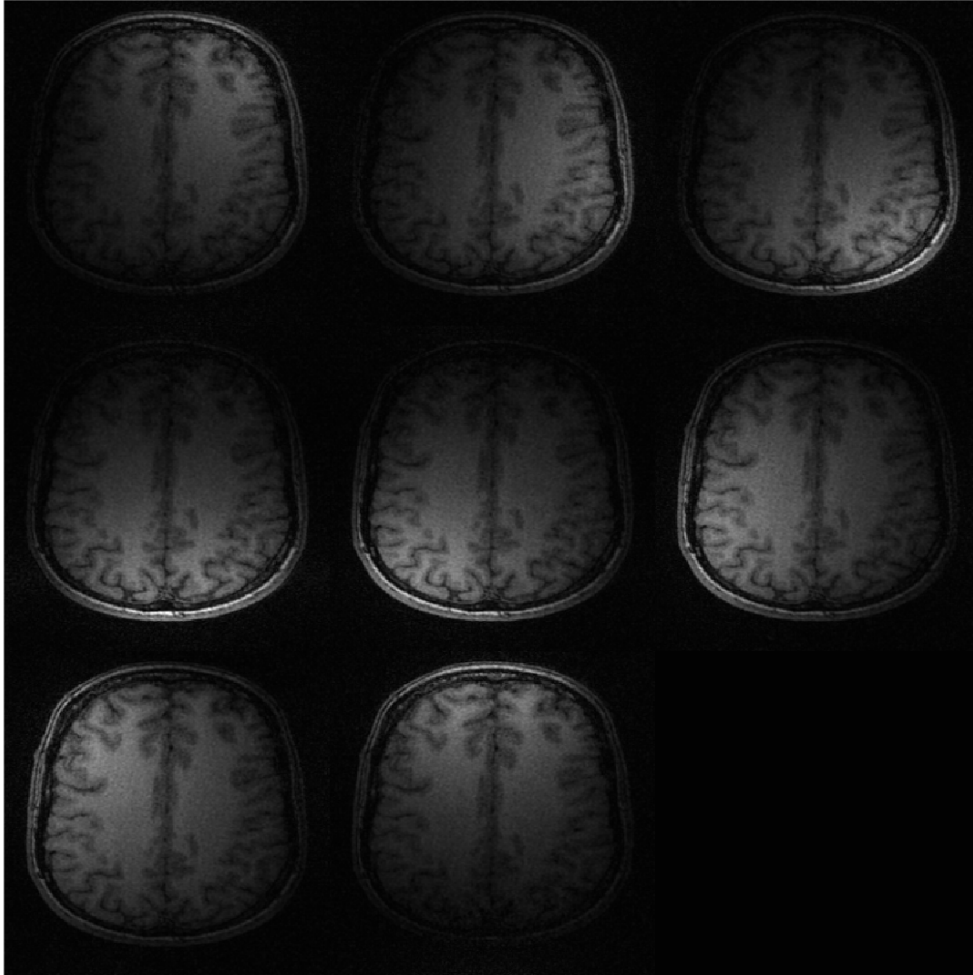


Figure 19: Coil images after the last SPIRiT iteration.

### Part 7: Last SPIRiT iteration k-space spectra

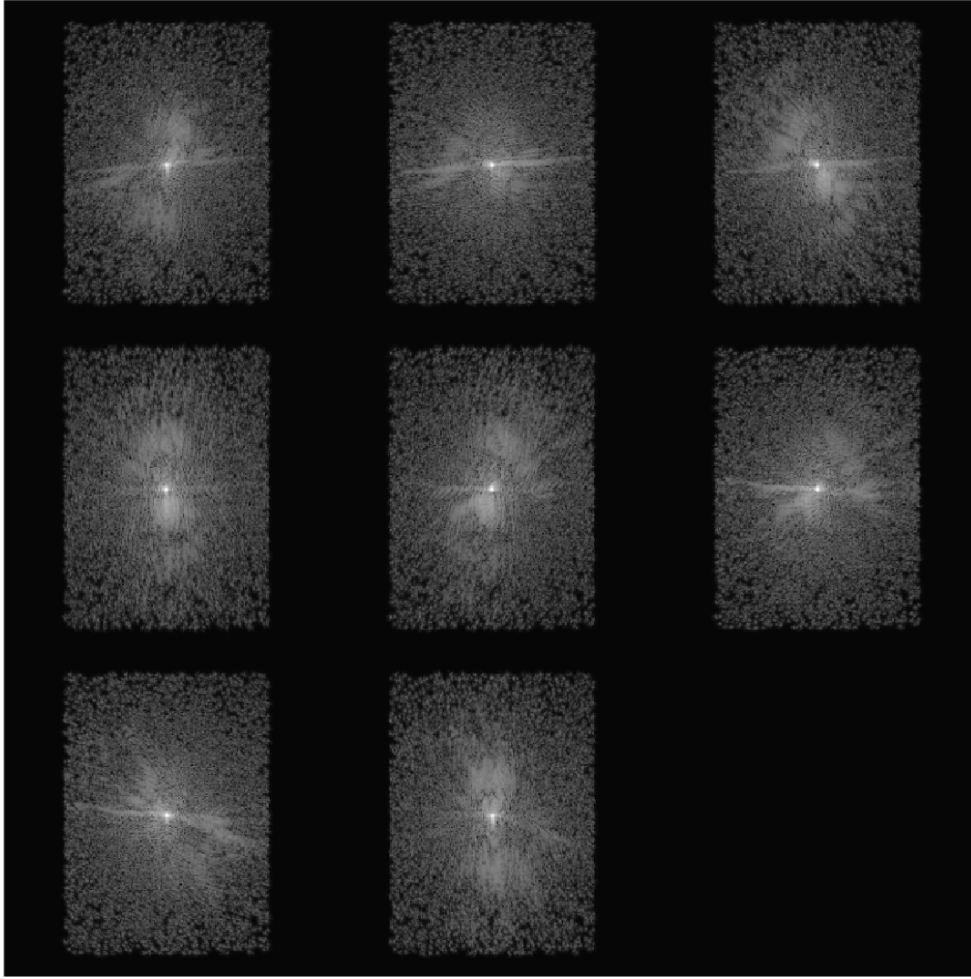
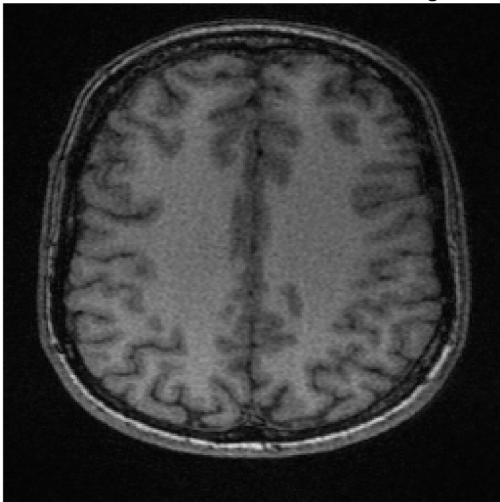


Figure 20: K-space spectra after the last SPIRiT iteration.

Part 7: Last SPIRiT iteration SoS image



Part 7: Last SPIRiT iteration absolute error

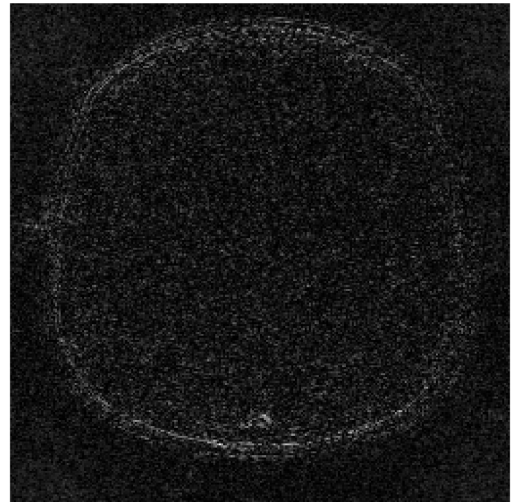


Figure 21: Last SPIRiT SoS image and error.

The metric curves show whether repeated kernel application converges or amplifies residual interpolation error. PSNR increases monotonically from 30.63 dB to 32.91 dB, which indicates

that repeated SPIRiT updates progressively improve the least-squares interpolation of missing k-space values. SSIM increases until approximately iteration 5, where it reaches 0.8713, and then decreases slightly even though PSNR continues to rise. This means that later iterations reduce global mean-square error but may also introduce mild structural smoothing or texture changes. Overall, the full SPIRiT reconstruction is substantially better than the zero-filled result and also better than the single-iteration result. With a small  $3 \times 3$  kernel, the improvement eventually saturates because the interpolation uses only a very local neighborhood, especially in outer k-space where random sampling is sparse.

#### Relevant code.

```

1 psnrSpirit = zeros(Niter, 1);
2 ssimSpirit = zeros(Niter, 1);
3 MrIter = Mu;
4 imIter = imu;
5
6 for it = 1:Niter
7     [~, MrTmp] = spirit(MrIter, kernel);
8     MrIter = applyDataConsistency(MrTmp, Mu, mask);
9     imIter = ifft2cStack(MrIter);
10
11     iterSos = normalizeByP95(sosCombine(imIter));
12     [psnrSpirit(it), ssimSpirit(it)] = computeQuality(iterSos, refSos, max_val);
13 end
14
15 errSpiritLast = abs(iterSos - refSos);
16 saveMetricPlot(psnrSpirit, 'PSNR_(dB)', fullfile(outDir, 'part7_psnr_iterations.png'), ...
17     'Part_7:_SPIRiT_PSNR_versus_iteration');
18 saveMetricPlot(ssimSpirit, 'SSIM', fullfile(outDir, 'part7_ssim_iterations.png'), ...
19     'Part_7:_SPIRiT_SSIM_versus_iteration');
```

## 9 Part 8: One Iteration of $\ell_1$ -SPIRiT

The  $\ell_1$ -SPIRiT iteration applies data consistency, one SPIRiT update, wavelet-domain soft-thresholding, and a centered FFT back to k-space. I used  $\beta = 0.0001$ . One iteration gives PSNR = 28.23 dB and SSIM = 0.7904.

## Part 8: One $\ell_1$ -SPIRiT iteration coil images

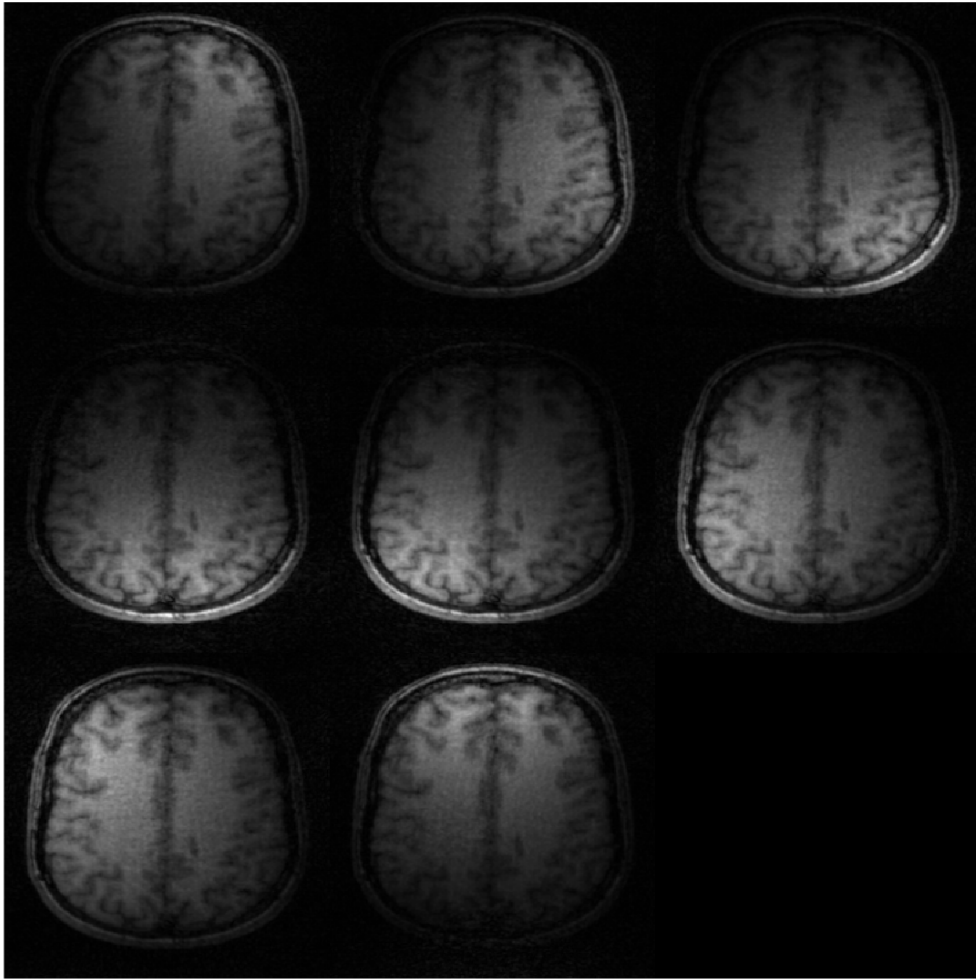


Figure 22: Coil images after one  $\ell_1$ -SPIRiT iteration.

### Part 8: One $\ell_1$ -SPIRiT iteration k-space spectra

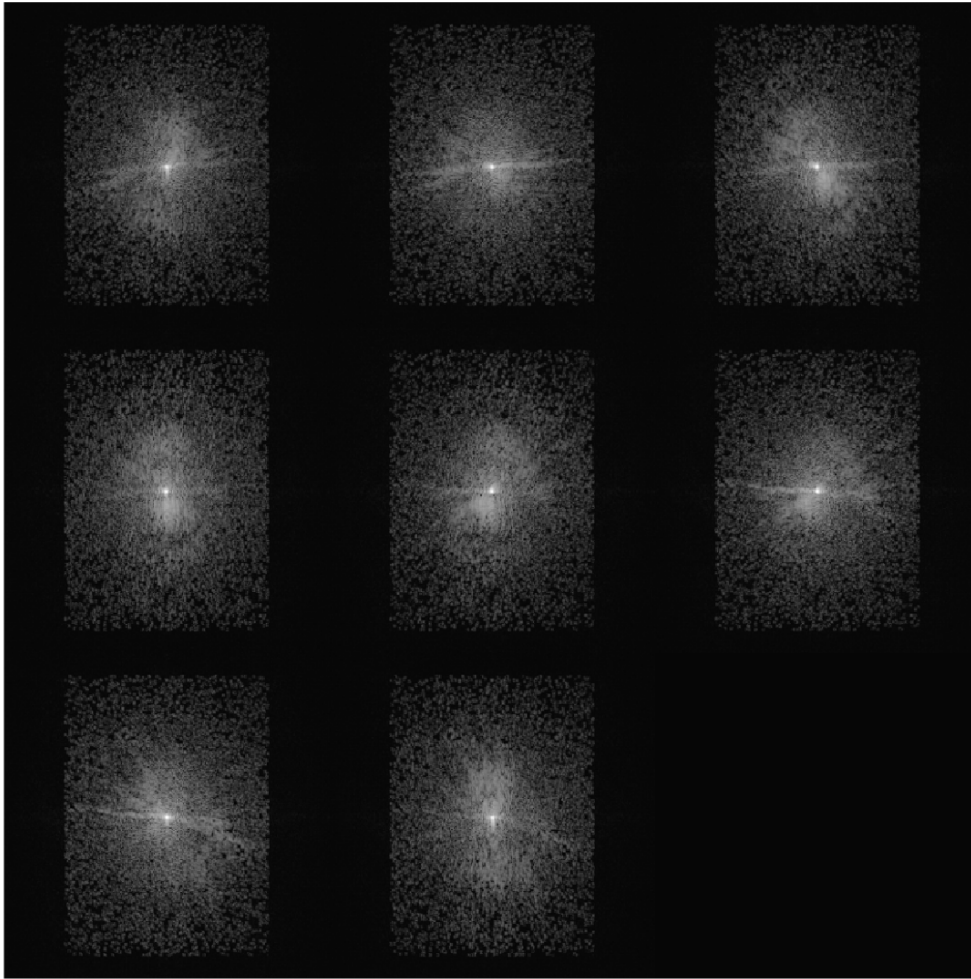
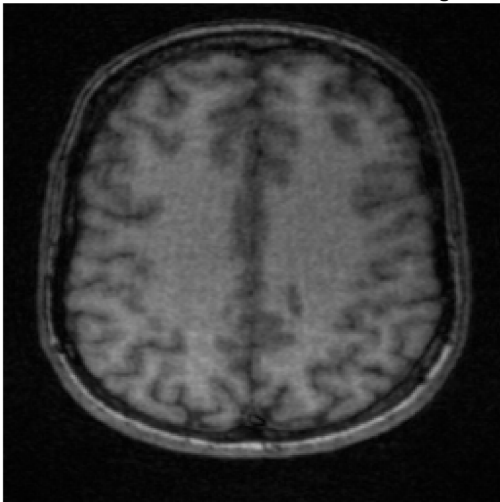


Figure 23: K-space spectra after one  $\ell_1$ -SPIRiT iteration.

Part 8: One  $\ell_1$ -SPIRiT iteration SoS image



Part 8: One  $\ell_1$ -SPIRiT iteration absolute error

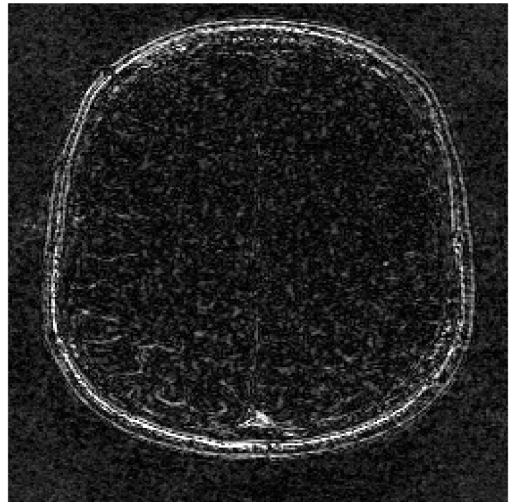


Figure 24: One-iteration  $\ell_1$ -SPIRiT SoS image and error.

The wavelet step adds a sparsity prior to the SPIRiT consistency prior. This helps suppress incoherent artifacts from random undersampling, but the threshold must be chosen carefully to

avoid oversmoothing.

#### Relevant code.

```

1 [imL1OneAll, ML1OneAll] = l1spirit(Mu, mask, kernel, betaL1Spirit, 1);
2 imL1One = imL1OneAll(:, :, :, 1);
3 ML1One = ML1OneAll(:, :, :, 1);
4 l1OneSos = normalizeByP95(sosCombine(imL1One));
5 [psnrL1One, ssimL1One] = computeQuality(l1OneSos, refSos, max_val);
6 errL1One = abs(l1OneSos - refSos);
7
8 function [imr, Mr] = l1spirit(Mu, mask, kernel, beta, Niter)
9     [Nx, Ny, Nc] = size(Mu);
10    Mr = zeros(Nx, Ny, Nc, Niter);
11    imr = zeros(Nx, Ny, Nc, Niter);
12    MrCur = Mu;
13
14    for it = 1:Niter
15        MrCur = applyDataConsistency(MrCur, Mu, mask);
16        [imSpirit, ~] = spirit(MrCur, kernel);
17
18        imTh = zeros(Nx, Ny, Nc);
19        for c = 1:Nc
20            imTh(:, :, c) = l1wavelet(imSpirit(:, :, c), beta);
21        end
22
23        MrCur = fft2cStack(imTh);
24        imr(:, :, :, it) = imTh;
25        Mr(:, :, :, it) = MrCur;
26    end
27 end

```

## 10 Part 9: Ten Iterations of $\ell_1$ -SPIRiT

For the final experiment,  $\ell_1$ -SPIRiT was run for 10 iterations using the same calibrated SPIRiT kernel and the same  $\beta = 0.0001$ . The last iteration gives PSNR = 31.54 dB and SSIM = 0.8101.

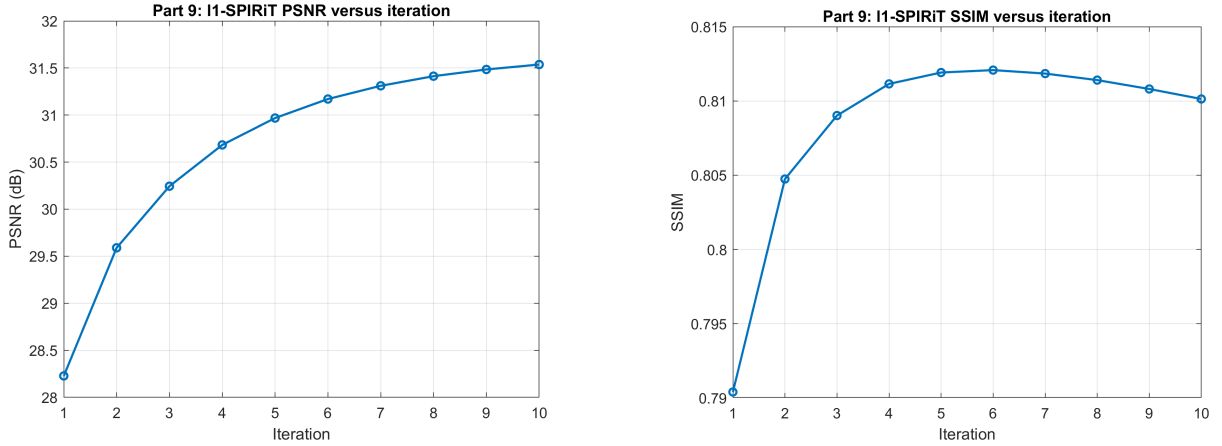


Figure 25: PSNR and SSIM versus  $\ell_1$ -SPIRiT iteration number.

| Iteration | $\beta$ | PSNR (dB) | SSIM   |
|-----------|---------|-----------|--------|
| 1         | 0.0001  | 28.23     | 0.7904 |
| 2         | 0.0001  | 29.59     | 0.8047 |
| 3         | 0.0001  | 30.24     | 0.8090 |
| 4         | 0.0001  | 30.68     | 0.8111 |
| 5         | 0.0001  | 30.97     | 0.8119 |
| 6         | 0.0001  | 31.17     | 0.8121 |
| 7         | 0.0001  | 31.31     | 0.8118 |
| 8         | 0.0001  | 31.41     | 0.8114 |
| 9         | 0.0001  | 31.48     | 0.8108 |
| 10        | 0.0001  | 31.54     | 0.8101 |

Table 3: Quantitative results for 10  $\ell_1$ -SPIRiT iterations.

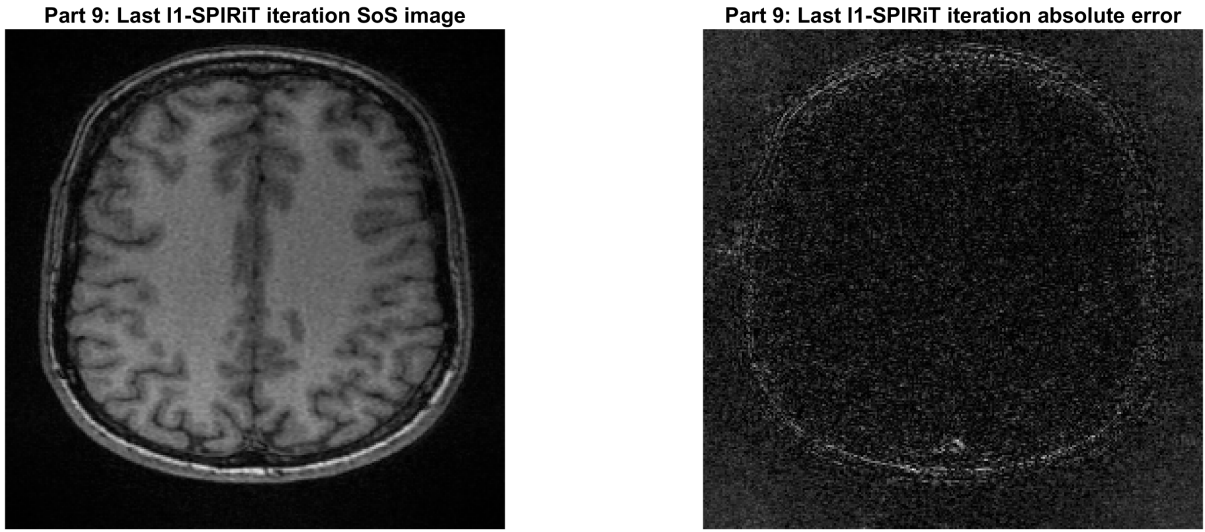


Figure 26: Last  $\ell_1$ -SPIRiT SoS image and error.

The  $\ell_1$ -SPIRiT reconstruction improves over the zero-filled image, but with the selected  $3 \times 3$  kernel and fixed  $\beta$  it remains below the SPIRiT-only result in PSNR and SSIM. PSNR increases monotonically from 28.23 dB to 31.54 dB, so the iterative process is still moving the reconstruction closer to the reference in a mean-square-error sense. SSIM, however, reaches its maximum around iteration 6 and then decreases slightly. This behavior is consistent with a constant wavelet threshold: repeated soft-thresholding keeps suppressing small coefficients, which can remove incoherent artifacts but can also reduce fine anatomical texture. The result suggests that a smaller  $\beta$ , a decreasing  $\beta$  schedule, or a larger SPIRiT kernel could improve the final balance between sparsity and structural fidelity.

#### Relevant code.

```

1 [imL1All, ML1All] = l1spirit(Mu, mask, kernel, betaL1Spirit, Niter);
2 psnrL1 = zeros(Niter, 1);
3 ssimL1 = zeros(Niter, 1);
4
5 for it = 1:Niter
6     l1SosIt = normalizeByP95(sosCombine(imL1All(:, :, :, it)));
7     [psnrL1(it), ssimL1(it)] = computeQuality(l1SosIt, refSos, max_val);
8 end
9
10 l1LastSos = normalizeByP95(sosCombine(imL1All(:, :, :, Niter)));
11 errL1Last = abs(l1LastSos - refSos);
12

```

```

13 saveMetricPlot(psnrL1, 'PSNR_(dB)', fullfile(outDir, 'part9_psnr_iterations.png'), ...
14     'Part9:  $\ell_1$ -SPIRiT PSNR versus iteration');
15 saveMetricPlot(ssimL1, 'SSIM', fullfile(outDir, 'part9_ssim_iterations.png'), ...
16     'Part9:  $\ell_1$ -SPIRiT SSIM versus iteration');
17 saveImageFigure(l1LastSos, [0 max_val], fullfile(outDir, 'part9_last_sos.png'), ...
18     'Part9:  $\ell_1$ -SPIRiT iteration SoS image');
19 saveImageFigure(errL1Last, [0 0.2*max_val], fullfile(outDir, 'part9_last_error.png'), ...
20     'Part9:  $\ell_1$ -SPIRiT iteration absolute error');

```

## 11 Conclusion

The experiment demonstrates the progression from zero-filled random undersampling to wavelet denoising, SPIRiT reconstruction, and iterative  $\ell_1$ -SPIRiT. Random undersampling produces incoherent artifacts, wavelet thresholding suppresses sparse-domain noise-like components, and SPIRiT uses multi-coil neighborhood consistency to estimate missing k-space values. In these results, iterative SPIRiT with data consistency gives the strongest quantitative performance, while  $\ell_1$ -SPIRiT still improves substantially over zero-fill. The final quantitative comparison is saved in `outputs/results_table.csv`, and the console log is saved in `outputs/console_output.txt`.

## 12 Shared Helper Functions

The main reconstruction functions were shown under their corresponding parts. The shared helper functions used for centered FFTs, SoS combination, normalization, data quality metrics, and zero-padded neighbor shifts are listed below.

```
1 function M = fft2cStack(im)
2     M = zeros(size(im));
3     for c = 1:size(im, 3)
4         M(:, :, c) = fft2c(im(:, :, c));
5     end
6 end
7
8 function im = ifft2cStack(M)
9     im = zeros(size(M));
10    for c = 1:size(M, 3)
11        im(:, :, c) = ifft2c(M(:, :, c));
12    end
13 end
14
15 function M = fft2c(im)
16     M = fftshift(fft2(ifftshift(im)));
17 end
18
19 function im = ifft2c(M)
20     im = fftshift(ifft2(ifftshift(M)));
21 end
22
23 function out = shift2dZero(in, dx, dy)
24     [Nx, Ny] = size(in);
25     out = zeros(Nx, Ny);
26
27     dstX = max(1, 1 - dx):min(Nx, Nx - dx);
28     dstY = max(1, 1 - dy):min(Ny, Ny - dy);
29     srcX = dstX + dx;
30     srcY = dstY + dy;
31
32     out(dstX, dstY) = in(srcX, srcY);
33 end
34
35 function sos = sosCombine(im)
36     sos = sqrt(sum(abs(im).^2, 3));
37 end
38
39 function imn = normalizeByP95(im)
40     p95 = prctile(abs(im(:)), 95);
41     if p95 == 0
42         imn = abs(im);
43     else
44         imn = abs(im) / p95;
45     end
46 end
47
48 function [p, s] = computeQuality(imTest, imRef, maxVal)
49     testMetric = double(imTest) / maxVal;
50     refMetric = double(imRef) / maxVal;
51     mseVal = mean((testMetric(:) - refMetric(:)).^2);
52     if mseVal == 0
53         p = Inf;
54     else
55         p = 10 * log10(1 / mseVal);
56     end
57     s = ssim(testMetric, refMetric, 'DynamicRange', 1);
58 end
```